Aerospike DX Demo Appliance

Overview & framework

This project involves creating a one-command, self-contained demo appliance whose primary job is to eliminate setup and recovery friction (one application, with each frontend action mapping to a specific Aerospike capability).

Six data-driven scenario definitions detail the main demo moments/actions:

- **Search near airport** (search geographically on the server)
- **Increase radius** (expand search radius)
- **Click neighborhood** (change neighborhood)
- **Filter rooms** (query nested data without client-side traversal)
- **Search name** (handle real-world Unicode data)
- **Book** (click book)
- **Inspect record** (in Voyager)

These definitions allow us to decouple the demo narrative from the backend, eliminating the need to hard-code the flow.

The UI, script, and cheat sheet all use the same scenario definitions.

Demo functionality

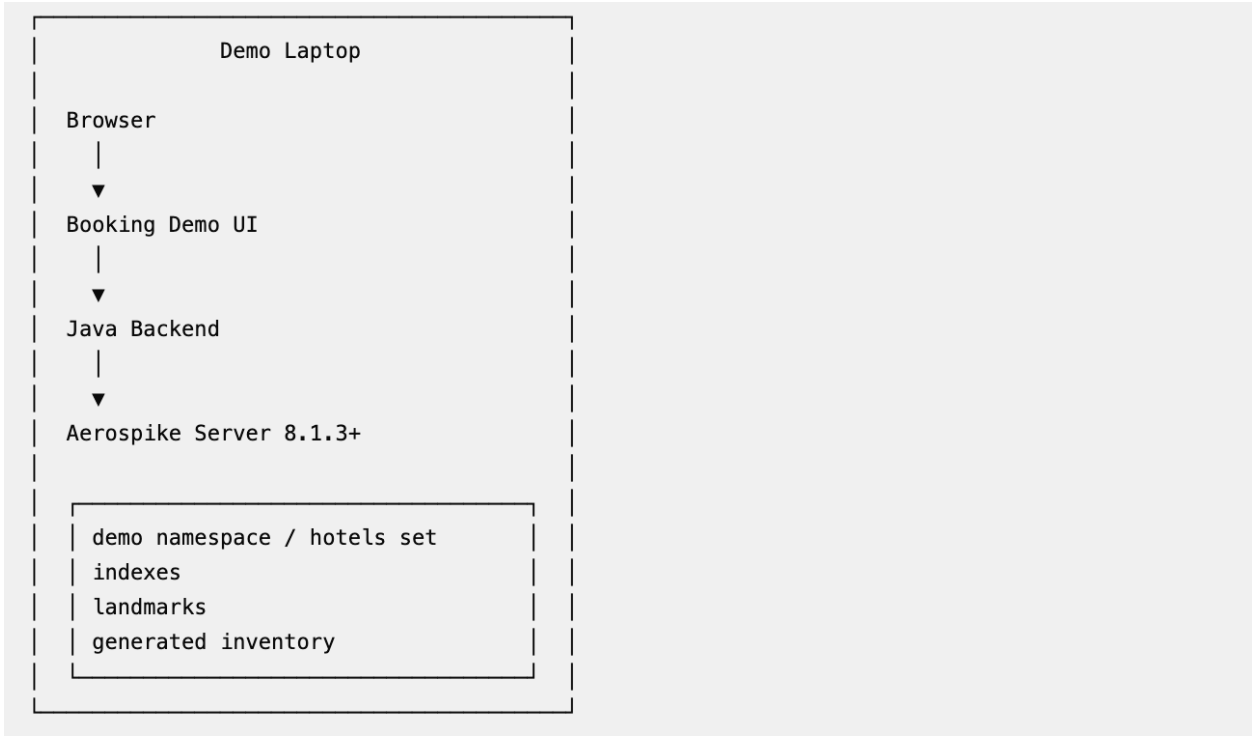
1. Entire demo packaged as Docker Compose

The SE runs:

```
None
git clone https://github.com/aerospike-examples/booking-demo

./demo start
```

The package launches:



SEs will be running this with prospects and at meetups, so the assumption is that there is no cloud/Aerospike account, no manual database installation, and no CLI configuration.

2. Single demo command

The demo will be packaged with a lightweight wrapper around Docker Compose:

```
None
./demo start
```

Then:

```
None
✓ Checking Docker
✓ Starting Aerospike
✓ Starting booking application
✓ Loading demo data
✓ Creating indexes
✓ Waiting for indexes → RW
✓ Starting web application
```

```
Demo ready!

Open:
http://localhost:8080

Demo controls:
  ./demo reset
  ./demo seed
  ./demo status
  ./demo logs
  ./demo stop
```

The RW check is critical. Idexes transition from WO to RW and shouldn't be queried until they're ready. `waitTillComplete()` should also be identified as an important DX advantage.

3. Demo resettable in 5 seconds (or less)

When the SE does:

```
None
./demo reset
```

The entire system is returned to a known state.

For example:

```
None
./demo reset

Resetting demo...
✓ Removed bookings
✓ Restored availability
✓ Restored demo data
✓ Rebuilt indexes
✓ Demo ready
```

If someone at a meetup (inevitably) asks to see the booking again, the SE shouldn't have to remember how to manually undo the booking.

The booking model is well-suited for this because booking is an atomic update that removes days from `available` and adds them to `booked`.

4. Building & invoking demo mode

The demo interface will include an unobtrusive **Demo Mode** button. When activated, the UI shows something like:

```
┌──────────────────────────────────────────────┐
│ SEARCH RESULTS                               │
│                                              │
│ Lakeside Grand                               │
│ ★★★★☆  8.6    $144                           │
│                                              │
│ [ View Rooms ]                               │
│                                              │
├──────────────────────────────────────────────┤
│ AEROSPIKE                                    │
│                                              │
│ Query                                        │
│ $.locality == 'austin-downtown'              │
│ and $.rating >= 80                           │
│                                              │
│ Index                                        │
│ hotel-locality-idx                           │
│                                              │
│ Query mechanism                              │
│ AEL + Secondary Index                        │
└──────────────────────────────────────────────┘
```

The purpose is to show the AEL query alongside the result and update it as filters change. This makes the query-language claim demonstrable rather than merely asserted.

Additionally, the panel shows:

What the user did

> "Search hotels near the airport"

What Aerospike did

> GEO2DSPHERE index → AEL filter → results

Relevant code

```
None
session.query(hotels)
    .where(...)
    .execute();
```

That gives the SE a natural transition. (e.g., *“You see this result? Let's look at what actually happened underneath."*)

5. Seed the database automatically

Instead of the SE running a separate data-generation script, the demo app does the following when starting up:

```
None
if database empty:
    generate data
    load data
    create indexes
    wait for indexes
```

And provide:

```
None
./demo reset
./demo reseed
```

The generator should be **deterministic**:

```
None
SEED=12345
```

That means every SE gets essentially the same:

- hotels
- ratings
- prices

- amenities
- room inventory
- availability
- search results

This makes screenshots, videos, training material, and troubleshooting much easier.

6. Separate presentation & engineering modes

The demo app will provide three UI levels:

Level 1: Customer mode

Aesthetically pleasing booking site with no technical clutter.

```
Where are you going?
Austin

Dates
Sep 16 – Sep 20

[ Search ]

312 hotels found
```

Level 2: Demo mode

When **Show Aerospike** is toggled, the UI shows:

```
Query
Index
AEL
Path Expression
Records examined
```

Level 3: Engineering mode

For technical audiences:

```
AEL
──────────────────
$.rooms.*.rates.*
  .available.*
  [?(@ >= 20260916
    and @ <= 20260919)]
  .count() == 4

Index
──────────────────
hotel-minprice-idx

Index state
──────────────────
RW

Query hint
──────────────────
forIndex(...)
```

This lets the same packaged application serve both a CIO-level meetup conversation and a deep technical discussion.

7. “Reset everything" emergency button

This will be directly in the UI:

↻ **Reset Demo**

with confirmation: “Reset the demo to its starting state?”

This way, SEs don’t have to open Terminal because something went sideways.

Package repository

The demo app will be packaged up as follows:

```
None
aerospike-booking-demo/
│
├── demo
```

```
├── docker-compose.yml
├── README.md
│
├── backend/
│   ├── src/
│   ├── pom.xml
│   └── Dockerfile
│
├── frontend/
│   ├── src/
│   └── Dockerfile
│
├── seed/
│   ├── generator
│   └── scenarios
│
├── aerospike/
│   ├── aerospike.conf
│   └── init/
│
├── demo/
│   ├── scenarios/
│   │   ├── geo.json
│   │   ├── locality.json
│   │   ├── path-expressions.json
│   │   ├── strings.json
│   │   └── booking.json
│   │
│   └── scripts/
│       ├── start
│       ├── reset
│       ├── seed
│       └── health
│
└── docs/
    ├── sales-playbook.md
```

```
├── technical-deep-dive.md
└── troubleshooting.md
```